Types of Methods:

Inside Python class 3 types of methods are allowed

1) Instance Methods

2) Class Methods

3) Static Methods

1) Instance Methods:

- Inside method implementation if we are using instance variables then such type of methods are called instance methods.
- Inside instance method declaration, we have to pass self variable.
  Ex:-> def m1(self):
- By using self variable inside method we can able to access instance variables.
- Within the class we can call instance method by using self variable and from outside of the class we can call by using object reference.

```
class Student:
    “”” Student Class “”””
    def __init__(self,name,marks):
        self.name=name
        self.marks=marks

    def display(self):
            print(f"Hi {self.name}")
            print(f"Your Marks are {self.marks}")

    def grade(self):
        if self.marks>=60:
            print('You got First Grade')
        elif self.marks>=50:
            print('Yout got Second Grade')
        elif self.marks>=35:
            print('You got Third Grade')
        else:
            print('You are Failed')

s = Student("prateek", 124)
s.display()
s.grade()
```

main.py

```python
class Student:

    def __init__(self,name,marks):
        self.name=name
        self.marks=marks

    def display(self):
            print(f"Hi {self.name}")
            print(f"Your Marks are {self.marks}")

    def grade(self):
        if self.marks>=60:
            print('You got First Grade')
        elif self.marks>=50:
            print('Yout got Second Grade')
        elif self.marks>=35:
            print('You got Third Grade')
        else:
            print('You are Failed')

s = Student("prateek", 124)
s.display()
s.grade()
```

Output

```
Hi prateek
Your Marks are 124
You got First Grade

=== Code Execution Successful ===
```

Above example I have shown for 1 student. If We need to do for multiple then we can do like below.

```python
n=int(input('Enter number of students:'))
for i in range(n):
    name=input('Enter Name:')
    marks=int(input('Enter Marks:'))
    s= Student(name,marks)
    s.display()
    s.grade()
    print()
```

Setter and Getter Methods:

We can set and get the values of instance variables by using getter and setter methods.

Setter Method:

setter methods can be used to set values to the instance variables.

setter methods also known as mutator methods.

Syntax:

```
def setVariable(self,variable):
        self.variable=variable
```

Example:

```
def setName(self,name):
        self.name=name
```

Getter Method:

Getter methods can be used to get values of the instance variables.

Getter methods also known as accessor methods.

Syntax:

```
def getVariable(self):
     return self.variable
```

Example:

```
def getName(self):
    return self.name
```

```
class Student:

    def setName(self,name):
        self.name=name

    def getName(self):
        return self.name

    def setMarks(self,marks):
        self.marks=marks

    def getMarks(self):
        return self.marks

n=int(input('Enter number of students:'))
for i in range(n):
    s=Student()
    name=input('Enter Name:')
    s.setName(name)
    marks=int(input('Enter Marks:'))
    s.setMarks(marks)
    print('Hi',s.getName())
    print('Your Marks are:',s.getMarks())
    print()
```

main.py

```python
class Student:

    def setName(self,name):
        self.name=name

    def getName(self):
        return self.name

    def setMarks(self,marks):
        self.marks=marks

    def getMarks(self):
        return self.marks

n=int(input('Enter number of students:'))
for i in range(n):
    s=Student()
    name=input('Enter Name:')
    s.setName(name)
    marks=int(input('Enter Marks:'))
    s.setMarks(marks)
    print('Hi',s.getName())
    print('Your Marks are:',s.getMarks())
    print()
```

Output

```
Enter number of students:2
Enter Name:prateek
Enter Marks:123
Hi prateek
Your Marks are: 123

Enter Name:pankaj
Enter Marks:345
Hi pankaj
Your Marks are: 345


=== Code Execution Successful ===
```

2) Class Methods:

- Inside method implementation if we are using only class variables (static variables), then such type of methods we should declare as class method.

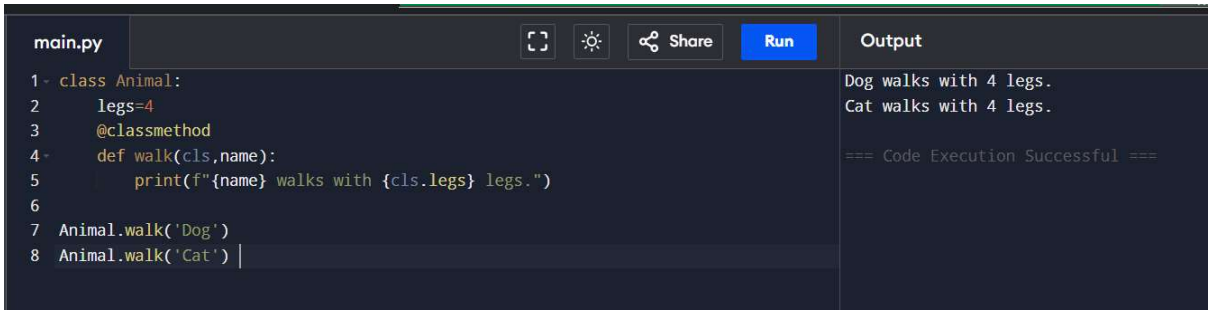
- We can declare class method explicitly by using @classmethod decorator.

- For class method we should provide cls variable at the time of declaration

- We can call classmethod by using classname or object reference variable.

```
class Animal:
    legs=4
    @classmethod
    def walk(cls,name):
        print(f"{name} walks with {cls.legs} legs.")

Animal.walk('Dog')
Animal.walk('Cat')
```



QUES:-

Program to track the Number of Objects created for a Class?

```
class Test:
    count=0
    def __init__(self):
        Test.count =Test.count+1

    @classmethod
    def no_of_objects(cls):
        print('The number of objects created for test class:',cls.count)

t1=Test()
t2=Test()
Test.no_of_objects()
t3=Test()
t4=Test()
t5=Test()
Test.no_of_objects()
```

main.py

```python
class Test:
    count=0
    def __init__(self):
        Test.count =Test.count+1

    @classmethod
    def no_of_objects(cls):
        print('The number of objects created for test class:',cls.count)

t1=Test()
t2=Test()
Test.no_of_objects()
t3=Test()
t4=Test()
t5=Test()
Test.no_of_objects()
```

Output

```
The number of objects created for test class: 2
The number of objects created for test class: 5

=== Code Execution Successful ===
```

3) Static Methods:

- In general these methods are general utility methods. Inside these methods we won't use any instance or class variables.

- Here we won't provide self or cls arguments at the time of declaration.

- We can declare static method explicitly by using @staticmethod decorator

- We can access static methods by using classname or object reference

```
class MathOperations:
    @staticmethod
    def add(x, y):
        """Adds two numbers, no instance or class state needed."""
        return x + y

# Call it directly on the class
result = MathOperations.add(5, 10)
print(result) # Output: 15

# You can also call it on an instance, but the instance is ignored
math_instance = MathOperations()
# result_from_instance
math_instance.add(5, 10)
```

Inner Classes

- Sometimes we can declare a class inside another class, such type of classes are called inner classes.

- Without existing one type of object if there is no chance of existing another type of object, then we should go for inner classes.

Example: **Without existing Car object there is no chance of existing Engine object. Hence Engine class should be part of Car class.**

```
class Car:
    .....
    class Engine:
    ......
```

Example: **Without existing university object there is no chance of existing Department object**

```
class University:
    .....
    class Department:
    ......
```

Example: **Without existing Human there is no chance of existin Head. Hence Head should be part of Human.**

```
class Human:
   class Head:
```

Note: **Without existing outer class object there is no chance of existing inner class object. Hence inner class object is always associated with outer class object.**

```
class Outer:
    def __init__(self):
        print("outer class object creation")

    class Inner:
        def __init__(self):
            print("inner class object creation")
        def m1(self):
            print("inner class method")

o = Outer()
i = o.Inner() # Or Outer.Inner()
i.m1()
```

Output

outer class object creation

inner class object creation

inner class method

1. Automatic Inner Object Creation

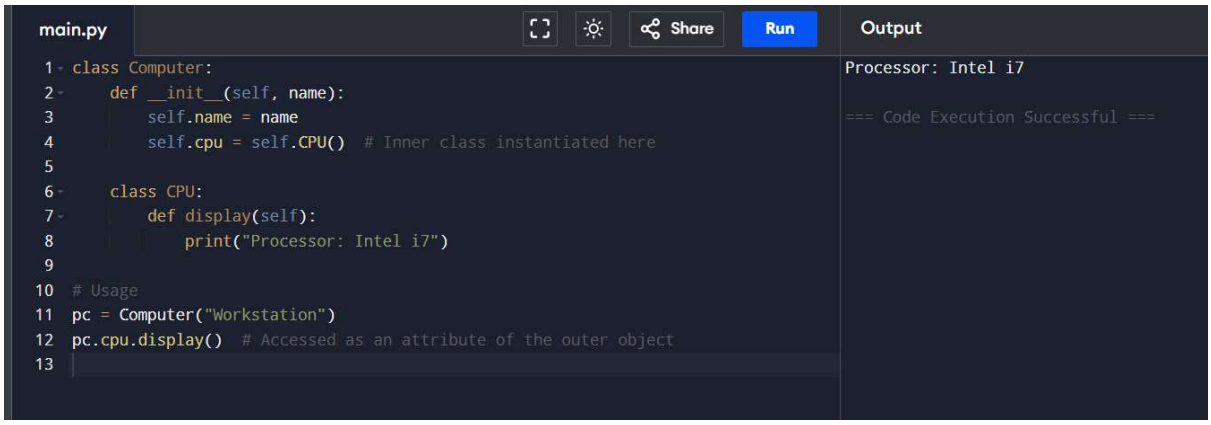
In this pattern, the outer class creates an instance of the inner class as soon as it is initialized. This is common when the inner class represents a "component" of the outer class.

EXAMPLE 1

```
class Computer:
    def __init__(self, name):
        self.name = name
        self.cpu = self.CPU()      # Inner class instantiated here

    class CPU:
        def display(self):
            print("Processor: Intel i7")

# Usage
pc = Computer("Workstation")
pc.cpu.display()  # Accessed as an attribute of the outer object
```



EXAMPLE 2

```
class Human:
    def __init__(self):
        self.name = 'Sunny'
        self.head = self.Head()
        self.brain = self.Brain()

    def display(self):
        print("Hello..", self.name)

    class Head:
        def talk(self):
            print('Talking...')

    class Brain:
        def think(self):
            print('Thinking...')

h = Human()
h.display()
h.head.talk()
h.brain.think()
```

main.py

```python
class Human:
    def __init__(self):
        self.name = 'Sunny'
        self.head = self.Head()
        self.brain = self.Brain()

    def display(self):
        print("Hello..", self.name)

    class Head:
        def talk(self):
            print('Talking...')

    class Brain:
        def think(self):
            print('Thinking...')

h = Human()
h.display()
h.head.talk()
h.brain.think()
```

Output

```
Hello.. Sunny
Talking...
Thinking...

=== Code Execution Successful ===
```

Garbage Collection

If an object does not have any reference variable, then that object eligible for Garbage Collection.

How to enable and disable Garbage Collector in our Program:

By default Gargbage collector is enabled, but we can disable based on our requirement. In this context we can use the following functions of gc module.

1) **gc.isenabled() ➔ Returns True if GC enabled**

2) **gc.disable()➔ To disable GC explicitly**

3) **gc.enable()➔ To enable GC explicitly**

```python
import gc
print(gc.isenabled())
gc.disable()
print(gc.isenabled())
gc.enable()
print(gc.isenabled())
```

Output

True
False
True

Destructors:

- Destructor is a special method and the name should be __del__
- Just before destroying an object Garbage Collector always calls destructor to perform clean-up activities (Resource deallocation activities like close database connection etc).
- Once destructor execution completed then Garbage Collector automatically destroys that object.

Note:

The job of destructor is not to destroy object and it is just to perform clean-up activities.

```
import time

class Test:
    def __init__(self):
        print("Object Initialization...")

    def __del__(self):
        print("Fulfilling Last Wish and performing clean up activities...")

t1 = Test()
t1 = None  # Reference count becomes 0; __del__ is called here

time.sleep(5)
```

```
print("End of application")
```

Why this works:

1. **Garbage Collection:** When you assign `t1 = None`, the `Test` object is orphaned. Python's garbage collector notices the **reference count** dropped to zero and executes the `__del__` method.
2. **Execution Order:** You will see the "clean up" message **before** the 5-second sleep begins because the object was destroyed manually at line 9.
3. **Module Import:** `import time` must be at the top level (left-aligned)